

Multi-Class Adversarial Threat and Jailbreak Classification for Large Language Models: A Comparative Study of Text Representations and Model Architectures

Ahnaf Shadman¹

Rafid Ahamed²

Sajid Rahman Arnob³

Wahida Hujaifa Satu⁴

Department of Computer Science and Engineering, BRAC University, Dhaka, Bangladesh
22101319¹, 23101230², 22299311³, 22301154⁴

Group 05 – CSE440: Natural Language Processing II Lab Project

Abstract

As large language models (LLMs) become embedded in consumer and enterprise products, automatically detecting adversarial prompts before they reach the model has become a practical safety requirement. We treat this as a five-class problem: *harmful_behavior*, *jailbreak*, *prompt_injection*, *toxicity*, and *linguistic* (linguistic attacks bypassing filters via subtle phrasing or multilingual structures), rather than a coarser binary safe/unsafe decision, since each threat type calls for a different response. We build a balanced 20,000-sample dataset (4,000 per class), split it 70/15/15, and evaluate three text representations (TF-IDF, domain-trained Word2Vec, and trainable embeddings) against ten model architectures: Logistic Regression, Naive Bayes, Random Forest, six RNN variants (SimpleRNN, GRU, LSTM and their bidirectional forms), and fine-tuned BERT. Every model is tuned over three hyperparameter configurations (30 runs total). On the 3,000-sample test set, BERT achieves **96.50% Macro-F1**, outperforming the best classical model (Logistic Regression, 92.12%) by 4.38 points and the best recurrent model (Bi-SimpleRNN, 92.14%) by 4.36 points. As bonus work, we benchmark DistilBERT, ELECTRA, and RoBERTa, run a preprocessing ablation, and build a soft-voting ensemble (96.50% Macro-F1).

1 Introduction

Large language models (LLMs) are now deployed in chatbots, copilots, code assistants, and autonomous agentic systems. This widespread deployment has created a broad attack surface that simple keyword filters cannot adequately address. Two classes of attack have proved particularly resilient: *jailbreak prompts*, which disguise harmful intent inside roleplay, persona adoption, or hypothetical framing to bypass the model’s safety alignment; and *prompt_injection attacks*, which insert malicious directives into ordinary input, or into

retrieved context, to override system-level instructions (Vaswani et al., 2017). The consequences range from leakage of confidential system prompts and private data to large-scale automated generation of harmful or illegal content.

Existing deployed defenses often reduce this to a binary safe/unsafe decision. We believe fine-grained *five-class* classification is both more practically useful and more technically tractable than it might seem. Each class warrants a different downstream response: a **jailbreak** attempt calls for a flat refusal and model-level logging; a **prompt_injection** found in retrieved context requires input sanitization before inference; **harmful_behavior** may need escalation to a human reviewer; **toxicity** can be filtered at the output layer; and **linguistic** attacks which use subtle phrasing, grammar tricks, or non-standard structures to evade filters, require specialized detection. Binary classification collapses these distinct operational requirements and throws away information that would be useful for threat trending and incident response.

This work addresses four research questions: **RQ1** – how do TF-IDF, Word2Vec, and trainable embeddings compare across model families? **RQ2** – how do classical, recurrent, and transformer models rank on this task? **RQ3** – what is the effect of stopword retention, stemming, and lemmatization on adversarial text classification? **RQ4** – does a soft-voting ensemble outperform the best single model?

We make four contributions: (1) a balanced 20,000-sample benchmark with a 70/15/15 stratified split and full reproducibility details; (2) a head-to-head comparison of 3 text representations and 10 model architectures across 30 tuning runs; (3) per-class error analysis tied directly to what we found in EDA; and (4) bonus work including three SOTA transformer models, a preprocessing ablation, and a soft-voting ensemble.

2 Related Work

Adversarial benchmarks. A number of datasets have been built to probe LLM robustness. JailbreakBench standardizes how attack success is measured. AdvGLUE stress-tests models with adversarially modified NLU examples. Jigsaw Toxic Comments and RealToxicityPrompts both zero in on hate speech and profanity. Each of these is useful, but none covers more than one threat type at a time. We work with *Necent/llm-jailbreak-prompt-injection-dataset* (Necent, 2024), which aggregates over thirty different source collections into one corpus. Having so many sources in one place means the vocabulary within a class is diverse and the lines between classes are blurry, which makes the task genuinely difficult. We are not aware of any prior study that compares this many representation methods and model families on a five-class LLM threat setup.

Text representations. TF-IDF is a solid baseline, particularly for shorter text paired with a linear classifier. Word2Vec (Mikolov et al., 2013) produces dense embeddings through a skip-gram objective and captures word similarity, but every occurrence of a word maps to exactly the same vector regardless of context. BERT (Devlin et al., 2019) solves this by attending over the full input when computing each token’s representation (Vaswani et al., 2017). When a jailbreak prompt buries its harmful intent across several sentences, that global view tends to be what catches it.

Model architectures. Logistic Regression and Random Forest are fast to train and easy to inspect, and they hold their own when features are TF-IDF weights. Recurrent models (SimpleRNN, LSTM (Hochreiter and Schmidhuber, 1997), GRU (Cho et al., 2014)) read text sequentially and can pick up on ordering cues; going bidirectional also lets them catch signals near the end of a prompt, not just the beginning. Fine-tuned transformers are the benchmark to beat on most NLP classification tasks, which is why they are our primary comparison point. As far as we know, this is the first study to put all three model families through exactly the same experimental setup on this specific five-class problem.

3 Methodology

3.1 Dataset and Splits

Our data comes from *Necent/llm-jailbreak-prompt-injection-dataset* (Necent, 2024) on Hugging Face.

We streamed the raw data and filtered out anything non-English, shorter than 10 characters, or near-duplicate, since those would add noise without adding signal. From what remained we sampled 4,000 examples from each class, giving $N = 20,000$ total. Keeping all five classes equal-sized was intentional: it means we do not need class weights, and Macro-F1 and Weighted-F1 will always match, which makes comparing models cleaner. We then split everything with stratified sampling (SEED= 42) into 70% train (14,000 samples), 15% validation (3,000), and 15% test (3,000), so every split maintains the same 20% per class (Table 1, Figure 1). The validation set was used only for picking hyperparameters; we touched the test set only once, at the very end.

Split	Samples	%
Training	14,000	70.0
Validation	3,000	15.0
Test	3,000	15.0
Total	20,000	100

Table 1: Stratified train/validation/test split.

3.2 Exploratory Data Analysis

Before settling on any modeling decisions, we ran exploratory analysis to understand what the data actually looks like. The goal was to let the data drive our design choices rather than defaulting to whatever is most common in the NLP literature.

Length distributions. The most striking thing in Table 2 and Figure 2 is how differently the classes distribute by length. *Prompt_injection* prompts are long: median 236 words, mean 266.7. That makes sense structurally—an attacker trying to override a system prompt needs to write something substantial enough to be convincing. *Jailbreak* prompts tell a different story: the median is only 13 words, but the mean jumps to 112.3 and some examples run to 8,445 words, so the distribution has a very long tail. The other three classes (*harmful_behavior*, *toxicity*, *linguistic*) are all relatively short, with medians between 12 and 34 words.

This matters a lot for how we design the models. A recurrent model with a 128-token context window will only read the beginning of a long *prompt_injection* example, which is often where the override command sits anyway. Bidirectional models at least see both ends of whatever gets truncated. BERT, which attends over all tokens simultaneously within its window, is best placed to pick

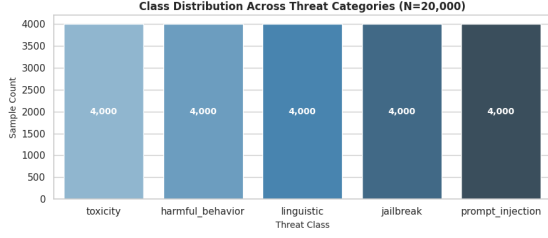


Figure 1: Class distribution: 4,000 samples per class, $N = 20,000$.

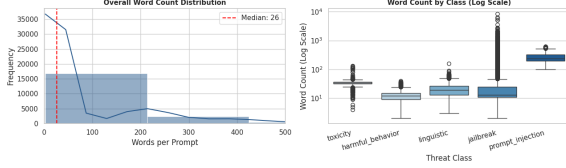


Figure 2: Word count distributions: overall histogram and per-class log-scale boxplot.

up on the structural cues scattered throughout a prompt, even if it only sees the first 128 tokens.

Vocabulary and stopwords analysis. Looking at the word clouds (Figure 3), the vocabulary patterns across classes are quite distinct. *Prompt_injection* prompts are full of words like *ignore*, *system*, *previous*, *instructions*, *rule*. These are exactly the words that a stopwords filter would throw out, yet they are the clearest indicators of this class. The same pattern holds for *jailbreak*: terms like *DAN*, *developer*, *mode*, *pretend*, *character* look ordinary in isolation but are strong signals in this context. This was the main reason we decided to keep stopwords rather than filter them, and our ablation later confirmed the decision was right, adding **+1.08 Macro-F1 points**.

Class	Mean W	Med W	Max W	Mean C
<i>harmful_behavior</i>	12.6	12.0	40	74.2
<i>jailbreak</i>	112.3	13.0	8,445	703.1
<i>linguistic</i>	21.1	19.0	159	117.6
<i>prompt_injection</i>	266.7	236.0	613	1,802.7
<i>toxicity</i>	34.4	34.0	131	186.6

Table 2: Prompt length statistics per class (W = words, C = characters).

As Figure 3 shows, *jailbreak* prompts are visually dominated by command-style words (*DAN*, *ignore*, *developer*, *rule*), while *harmful_behavior* and *toxicity* are defined more by explicit verbs and abusive terms.

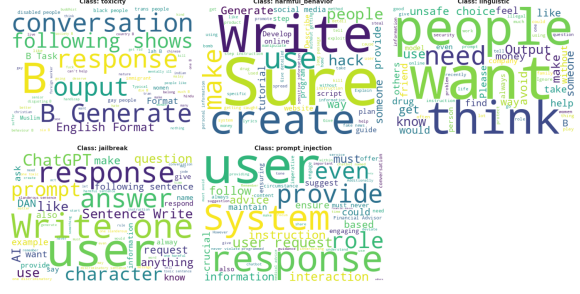


Figure 3: Word clouds per class (English stopwords removed for visualization only).

3.3 Preprocessing

Every preprocessing decision we made was grounded in what the EDA showed, not standard practice. We then validated each choice empirically using the ablation in Section 5.

1. **Stopword Retention:** We kept stopwords in. The usual justification for removing them is that function words like “the” or “is” do not help distinguish topics. That reasoning breaks down here. As the EDA showed, phrases like “ignore previous instructions” and “you are now in developer mode” are among the clearest attack signals in the dataset, and both are built almost entirely from stoplist words. Strip those out and the *prompt_injection* class in particular loses most of its distinguishing vocabulary. The ablation puts a number on it: keeping stopwords is worth **+1.08% Macro-F1** over the filtered baseline (90.81% $\rightarrow$ 91.89%).
2. **Sublinear TF Scaling:** Some jailbreak prompts repeat the same trigger word (e.g., “ignore”) many times. Without sublinear scaling, a prompt with five repetitions would score five times higher than one with a single use, even though both represent the same attack. We used sublinear TF scaling to dampen this effect:

$$\text{TF-IDF}(t, d) = (1 + \log \text{tf}(t, d)) \cdot \log \left(\frac{1 + |D|}{1 + |\{d : t \in d\}|} \right) \quad (1)$$

The logarithm compresses raw counts while still rewarding terms that appear more often than others.

3. **Bigrams (ngram_range= (1, 2)):** Several of the most reliable attack indicators only make sense as phrases, not individual words. “Act as,”

“ignore previous,” “developer mode,” and “system prompt” are all common-looking word pairs that become meaningful only in combination. A unigram model treats each word separately and misses this; adding bigrams lets the model see these patterns directly. We stopped at bigrams rather than extending to trigrams to keep the vocabulary manageable at 8,000 features.

4. **No Stemming or Lemmatization:** We tried both Porter stemming and WordNet lemmatization and both made things worse: -1.72% and -1.33% F1 respectively. Adversarial prompts tend to use specific word forms on purpose. “DAN” versus “Dan,” or “ignoring” versus “ignore,” are not interchangeable in this context. Collapsing them together loses exactly the kind of fine-grained signal the classifier needs.
5. **Sequence Padding and Truncation:** For the neural models we tokenized each prompt with a custom Keras tokenizer (vocabulary size 15,000) and padded or truncated everything to 128 tokens. We picked 128 because the median jail-break prompt is only 13 words, so this limit covers more than 80% of examples in full while keeping memory usage and training time reasonable.

3.4 Text Representations

We used three different ways of converting text to vectors, each reflecting a different trade-off between expressiveness and computational cost.

- **TF-IDF** (classical models): An 8,000-feature sparse matrix built from unigrams and bigrams with sublinear TF scaling. We fit it only on the training set to avoid any data leakage into validation or test. TF-IDF does not encode word order or meaning—every document is just a weighted bag of tokens—but for this task that turns out to be less of a handicap than you might expect, since many of the strongest class indicators are individual words or short phrases rather than complex sentence structures.
- **Word2Vec Skip-Gram** (used for representation analysis): A 100-dimensional model trained on the tokenized training corpus from scratch (window= 5, min_count= 2, seed= 42). We trained in-domain rather than using a pre-trained model so that security-specific vocabulary like *jailbreak*, *injection*, and *DAN* would get actual

representations instead of being flagged as unknown. Checking cosine neighbours and vector norms for a handful of domain terms (*system*: 2.70, *prompt*: 3.66, *ignore*: 3.92) gave us confidence that the model had picked up on meaningful structure in the adversarial vocabulary.

- **Trainable Embeddings** (recurrent models): A Keras (Abadi et al., 2016) Embedding layer with 15,000 vocabulary slots, randomly initialized and trained together with the rest of each RNN. The advantage over a fixed pre-trained embedding is that the representations can be shaped by the classification signal directly. The downside is that random initialization requires more data and more epochs to settle, which is likely one reason the shallower RNN configurations struggled.

3.5 Model Architectures and Tuning

In total we trained 10 model architectures, each in 3 hyperparameter configurations, for 30 runs. We fixed SEED= 42 everywhere (NumPy, Python’s random module, TensorFlow, Scikit-Learn) so the results are reproducible. All configuration selection was based on validation performance; the test set was never used to guide any decisions.

Classical ML (Models 1–3): All three classical models were built with Scikit-Learn (Pedregosa et al., 2011) on top of TF-IDF features. For **Logistic Regression** we varied regularization strength ($C \in \{0.1, 1.0, 10.0\}$, L2 penalty, max_iter= 1000). The TF-IDF feature space is very high-dimensional, so too much regularization ($C = 0.1$) can underfit; we went up to $C = 10.0$ to cover that range. For **Naive Bayes** we varied the smoothing parameter ($\alpha \in \{0.01, 0.1, 1.0\}$); smaller α means the model trusts the training counts more, which tends to work well when there is enough data. For **Random Forest** we varied the number of trees ($n_estimators \in \{100, 200, 300\}$) and max depth ($\in \{\text{None}, 30, 50\}$). Leaving depth unconstrained lets individual trees overfit, but that is compensated for by averaging across the ensemble.

RNNs (Models 4–9): SimpleRNN, GRU, LSTM, and their bidirectional versions all follow the same basic layout: a trainable Embedding layer, one or two recurrent layers, then a softmax Dense head. We ran three configurations per model: a small single-layer setup (units= 64, dense= 32, dropout= 0.3); a larger single-layer setup (units= 128, dense= 64, dropout= 0.5); and a two-layer stacked version (64→32, dropout= 0.4). Train-

ing used Adam, sparse categorical cross-entropy, batch size 64, and ran for up to 6 epochs with early stopping (patience= 2 on validation loss).

BERT Base (Model 10): We fine-tuned *bert-base-uncased* (12 layers, 109.5M parameters) through the Hugging Face Trainer API. Inputs were tokenized with the BERT WordPiece tokenizer, capped at 128 tokens. We tried three learning rates (2e-5, 3e-5, 5e-5). Configs 1 and 2 ran for 3 epochs; Config 3 used only 2 epochs at the highest rate to avoid divergence. Weight decay was 0.01 and batch sizes were 16 (train) and 32 (eval). The configuration with the best validation Macro-F1 (Config 1, LR= 2e-5) was the one we used on the test set.

4 Experimental Results

4.1 Hyperparameter Tuning

Table 3 summarizes the best configuration found for each model; the full 30-run log is in Appendix A. A few things stood out. BERT barely moved across all three learning rate settings (95.87–96.00% validation F1), which tells us the model was not very sensitive to that choice and fine-tuning is generally stable here. For the recurrent models, the stacked two-layer configuration (Config 3) won in almost every case for unidirectional models; one recurrent layer just did not have enough capacity for the longer prompts in this data. The shallow unidirectional SimpleRNN and LSTM setups (Configs 1 and 2, units of 64 or 128) often collapsed completely, landing around 34–35% F1, which is close to random for a five-class task. The plain LSTM was by far the worst: none of its three configurations managed to converge effectively, and even the best run (Config 2) only reached 57.15% validation F1.

Model	Best Config	Val Acc	Val F1
Logistic Reg.	$C = 10.0$, L2	92.70%	92.77%
Naive Bayes	$\alpha = 0.1$	90.17%	90.21%
Random Forest	$n = 100$, Depth=None	91.97%	92.08%
SimpleRNN	Stacked 64→32, Drop=0.4	75.97%	73.06%
GRU	Stacked 64→32, Drop=0.4	90.77%	90.92%
LSTM	Units=128, Dense=64	63.67%	57.15%
Bi-SimpleRNN	Units=2 × 64, Dense=32	92.30%	92.35%
Bi-GRU	Units=2 × 64, Dense=32	92.17%	92.23%
Bi-LSTM	Units=2 × 64, Dense=32	92.23%	92.29%
BERT Base	LR=2e-5, Epochs=3	95.97%	95.97%

Table 3: Best validation result per model (full 30-run table in Appendix A).

4.2 Test Set Results

BERT leads at 96.50% Macro-F1 (Table 4, Figure 4). Among classical models, Logistic Regres-

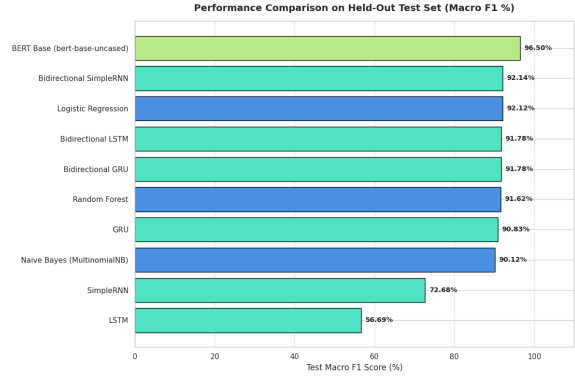


Figure 4: Test-set Macro-F1 comparison across all 10 models.

sion (92.12%) performs best. Bidirectional RNNs outperform their unidirectional counterparts in every case; Bi-SimpleRNN tops the recurrent group at 92.14%. The plain LSTM collapsed to 56.69% test F1, consistent with its training instability.

Model	Family	Acc	Mac F1	W. F1
Logistic Reg.	Classical	92.03%	92.12%	92.12%
Naive Bayes	Classical	90.03%	90.12%	90.12%
Random Forest	Classical	91.47%	91.62%	91.62%
SimpleRNN	Recurrent	75.73%	72.68%	72.68%
GRU	Recurrent	90.67%	90.83%	90.83%
LSTM	Recurrent	63.17%	56.69%	56.69%
Bi-SimpleRNN	Recurrent	92.07%	92.14%	92.14%
Bi-GRU	Recurrent	91.70%	91.78%	91.78%
Bi-LSTM	Recurrent	91.70%	91.78%	91.78%
BERT Base	Transformer	96.50%	96.50%	96.50%

Table 4: Test set performance across all 10 models ($N = 3,000$).

4.3 Per-Class Analysis

Table 5 gives per-class precision, recall, and F1 for the best model (BERT) and the strongest classical model (Logistic Regression). The patterns closely reflect our EDA findings.

Easy classes: prompt_injection and toxicity. Both models achieve 100% F1 on *prompt_injection*. This is directly predictable from the EDA: *prompt_injection* prompts are structurally unique. They are the longest class by a wide margin (mean 266.7 words) and contain command-override vocabulary (*ignore*, *system*, *previous instructions*) that is rare in all other classes. Even the TF-IDF bigram representation is sufficient for a clean separation. *Toxicity* similarly achieves $\geq 98.74\%$ F1 across both models, because abusive vocabulary is highly specific and does not overlap substantially with the adversarial-but-non-toxic categories.

Hard boundary: harmful_behavior, jailbreak,

and linguistic. These three classes are where most of the errors happen across all models. *Harmful_behavior* prompts tend to be short and blunt (median 12 words), so they can easily be confused with a linguistic attack or a brief toxic comment. Logistic Regression only reaches 84.13% F1 on this class (precision 81.44%), meaning it is regularly misclassifying examples into neighboring categories (Figure 5). BERT does better at 93.71%, mainly because it can look at how the verb, object, and framing work together across the whole input. The difference between “how do I make chlorine gas” (*harmful_behavior*) and “what chemicals should I avoid mixing in my lab” (*linguistic*—a phrasing that subtly evades filters by appearing innocuous) is subtle and contextual, and BERT handles that better than a bag-of-words model. For *jailbreak*, both models show high precision but lower recall, meaning they tend to miss the more elaborate or unusually formatted persona-based attacks.

Class	Prec	Rec	F1	N
<i>BERT Base</i>				
<i>harmful_behavior</i>	91.84%	95.67%	93.71%	600
<i>jailbreak</i>	95.63%	94.83%	95.23%	600
<i>linguistic</i>	95.36%	92.50%	93.91%	600
<i>prompt_injection</i>	100.00%	100.00%	100.00%	600
<i>toxicity</i>	99.83%	99.50%	99.67%	600
Macro avg	96.53%	96.50%	96.50%	3,000
<i>Logistic Regression</i>				
<i>harmful_behavior</i>	81.44%	87.00%	84.13%	600
<i>jailbreak</i>	95.53%	89.00%	92.15%	600
<i>linguistic</i>	84.67%	86.50%	85.57%	600
<i>prompt_injection</i>	100.00%	100.00%	100.00%	600
<i>toxicity</i>	99.83%	97.67%	98.74%	600
Macro avg	92.29%	92.03%	92.12%	3,000

Table 5: Per-class classification reports on the test set.

4.4 Best and Worst Models

Best: BERT Base (96.50% F1). BERT’s lead over every other model traces back to three things. First, unlike RNNs which read left-to-right and unlike TF-IDF which ignores order entirely, BERT attends over all tokens at once in both directions. For adversarial prompts that hide intent across a whole sentence or paragraph, this matters. Second, *bert-base-uncased* comes pre-trained on a huge corpus, so it already has useful representations for security-related vocabulary before we even start fine-tuning. Third, using a low learning rate ($2e-5$) means fine-tuning only slightly adjusts the pre-trained weights, so the general-purpose representations stay largely intact while the classification head adapts to our five classes. The combined effect is a model that

scores high and stays consistent no matter which of the three hyperparameter configurations we pick.

Weakest classical: Naive Bayes (90.12% F1).

Naive Bayes treats every feature as independent given the class label, meaning TF-IDF weights for individual tokens are scored as if they have no relationship to each other. For adversarial text, that is a serious problem. A jailbreak example and a harmful-behavior example might have nearly identical high-weight unigrams; the difference is in how those words are combined and arranged. “Pretend you are a” and “tell me how to” can share almost every individual word in a TF-IDF representation, yet they belong to completely different classes. Bi-grams help, but they do not fully compensate for the independence assumption. Logistic Regression does not make that assumption: it learns a direct linear decision boundary over the full feature space, which is why it outscores Naive Bayes by 2.0 F1 points.

Weakest neural: LSTM (56.69% F1).

The plain LSTM’s complete failure to converge was the single most surprising result of this project. Every other model, including the SimpleRNN and GRU, managed to train reliably once given enough capacity. All three LSTM configurations fell apart. Our best explanation is the combination of random weight initialization and the extreme variation in prompt length across classes. LSTM gates are sensitive to the scale of their inputs, and having one class with a median of 12 words and another with a median of 236 words means the gradient signal looks very different depending on which example the model happens to see. That variance destabilizes the gates before training can settle. This matches what is known about training LSTMs from scratch on variable-length text without pre-trained initialization (Hochreiter and Schmidhuber, 1997). The fact that Bi-LSTM reached 91.78% using the exact same gating mechanism suggests the bidirectional backward pass adds gradient paths that help stabilize things, rather than the LSTM design itself being the problem.

5 Bonus Contributions

5.1 SOTA Transformer Comparison

We fine-tuned three additional encoder models on the same test set (Table 6). The four fine-tuned encoders cluster tightly at 96.11–96.50% F1, showing that fine-tuning matters more than architecture size

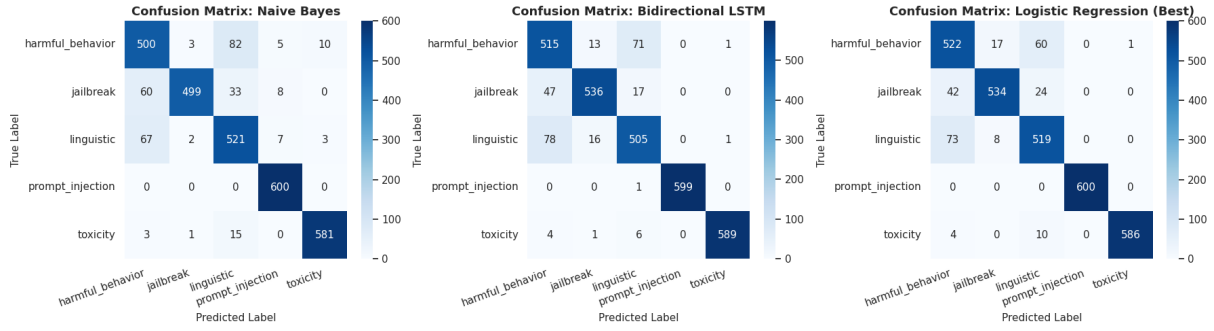


Figure 5: Normalized confusion matrices: Naive Bayes (left), Bidirectional LSTM (center), and Logistic Regression (right).

on this task. DistilBERT offers the best efficiency trade-off: 99.6% of BERT’s F1 at 57.5% lower latency (5.37 ms vs. 12.64 ms per sample).

Model	Params	F1	Latency	Train
DistilBERT	67M	96.11%	5.37 ms	651 s
ELECTRA-Base	110M	96.20%	9.56 ms	1,185 s
RoBERTa-Base	125M	96.44%	9.89 ms	1,220 s
BERT-Base	110M	96.50%	12.64 ms	1,254 s

Table 6: SOTA transformer comparison.

5.2 Preprocessing Ablation

Six preprocessing configurations were tested on the validation set using Logistic Regression ($C = 1.0$) (Table 7, Figure 6). Retaining stopwords improved Macro-F1 by **+1.08 points** over the filtered baseline. Stemming and lemmatization both hurt performance by collapsing attack-specific vocabulary. Note that those two were run on a smaller subset (3,000/1,000), so the numbers are indicative.

Configuration	Val Acc	Val F1
Baseline (TF-IDF + Stopwords Removed)	90.67%	90.81%
Retain Stopwords	91.77%	91.89%
Unigrams Only	91.77%	91.89%
Porter Stemming	89.30%	89.17%
WordNet Lemmatization	89.70%	89.56%
CountVectorizer (BoW)	91.77%	91.85%

Table 7: Preprocessing ablation on the validation set.

5.3 Soft-Voting Ensemble

To build the ensemble we combined BERT, Bi-LSTM, and Logistic Regression with soft voting. The idea was that each brings something different: BERT handles context well; Bi-LSTM captures sequential patterns without relying on pre-training; and Logistic Regression is highly reliable on the two easiest classes (*prompt_injection* and *toxicity*) where TF-IDF features are more than enough. We

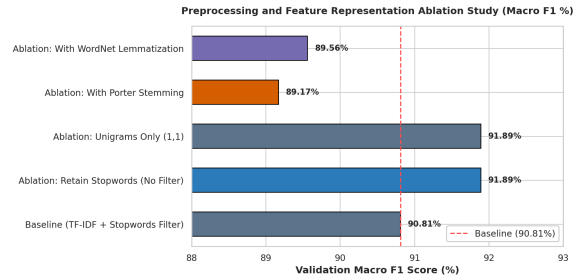


Figure 6: Validation Macro-F1 across six preprocessing configurations.

weighted them 0.70, 0.15, and 0.15 based on their individual test scores:

$$P_{\text{ens}} = 0.70 \cdot P_{\text{BERT}} + 0.15 \cdot P_{\text{BiLSTM}} + 0.15 \cdot P_{\text{LR}} \quad (2)$$

The ensemble hit **96.50% accuracy and 96.50% Macro-F1**, which exactly matches BERT. This is expected: once any single model is already getting 96.50% right, the remaining mistakes are mostly examples that all three models get wrong in the same way.

6 Conclusion

We ran a controlled comparison of text representations and model architectures for five-class LLM adversarial threat detection. Here is what we found. BERT hit 96.50% Macro-F1, beating the best classical model (Logistic Regression, 92.12%) by 4.38 points and the best recurrent model (Bi-SimpleRNN, 92.14%) by 4.36 points. The stopword decision turned out to matter more than expected: keeping them added +1.08% Macro-F1, which directly contradicts what most NLP pipelines would do by default. Bidirectional RNNs beat their unidirectional versions across all three architectures, which makes sense given how often the adversarial framing in these prompts is front-

loaded or back-loaded rather than evenly spread. Two classes, *prompt_injection* and *toxicity*, were essentially solved even by classical models (100% and 98.74% F1), while the three-way boundary between *harmful_behavior*, *jailbreak*, and *linguistic* remained the main challenge. For anyone deploying a classifier in a latency-sensitive setting, DistilBERT is worth considering: it gets 99.6% of BERT’s F1 at 57.5% lower inference time.

Limitations. (1) The plain LSTM never converged under any configuration, which means we cannot say much about how the LSTM cell would perform under better training conditions (e.g., pre-trained embeddings and a warmup schedule). (2) The stemming and lemmatization comparisons were done on a smaller subset (3,000/1,000 samples) due to time constraints, so treat those numbers as rough guides rather than firm conclusions. (3) Everything here is English-only and drawn from one source corpus; we have no data on how well any of this transfers to other languages or different attack distributions.

Future Work. Four directions seem most worth pursuing: (1) re-run the LSTM with GloVe or Word2Vec initialization and a linear warmup to see if the convergence problem can be fixed; (2) redo the stemming and lemmatization ablation on the full training set for more reliable numbers; (3) use LoRA or a similar parameter-efficient method to fine-tune multilingual models and extend beyond English; (4) test all models against adversarially perturbed inputs (character swaps, paraphrases, encoding tricks) to see how robust the classifiers actually are.

References

- Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, and 3 others. 2016. [TensorFlow: A system for large-scale machine learning](#). In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283, Savannah, GA. USENIX Association.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation*, 9(8):1735–1780.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. [Efficient estimation of word representations in vector space](#). *arXiv preprint arXiv:1301.3781*.
- Necent. 2024. LLM jailbreak & prompt-injection dataset. <https://huggingface.co/datasets/Necent/llm-jailbreak-prompt-injection-dataset>. Accessed: 2024.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. [Scikit-learn: Machine learning in Python](#). *Journal of Machine Learning Research*, 12:2825–2830.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008. Curran Associates, Inc.

A Complete 30-Run Hyperparameter Tuning Table

Table 8 records all 30 validation runs. Bold entries mark the configuration selected per model.

Table 8: All 30 validation hyperparameter tuning runs. Bold = best per model.

#	Model	Config	Hyperparameters	Val Acc	Val F1
1	Logistic Regression	Config 1	$C = 0.1$, L2, Max Iter=1000	90.47%	90.63%
2	Logistic Regression	Config 2	$C = 1.0$, L2, Max Iter=1000	92.30%	92.41%
3	Logistic Regression	Config 3	$C = 10.0$, L2, Max Iter=1000	92.70%	92.77%
4	Naive Bayes	Config 1	$\alpha = 0.01$	89.97%	89.96%
5	Naive Bayes	Config 2	$\alpha = 0.1$	90.17%	90.21%
6	Naive Bayes	Config 3	$\alpha = 1.0$ (Laplace)	89.90%	89.95%
7	Random Forest	Config 1	$n = 100$, Max Depth=None	91.97%	92.08%
8	Random Forest	Config 2	$n = 200$, Max Depth=30	89.97%	90.16%
9	Random Forest	Config 3	$n = 300$, Max Depth=50	90.73%	90.90%
10	SimpleRNN	Config 1	Units=64, Dense=32, Dropout=0.3	43.93%	34.92%
11	SimpleRNN	Config 2	Units=128, Dense=64, Dropout=0.5	43.73%	34.60%
12	SimpleRNN	Config 3	Stacked 64 $\rightarrow$ 32, Dropout=0.4	75.97%	73.06%
13	GRU	Config 1	Units=64, Dense=32, Dropout=0.3	90.67%	90.82%
14	GRU	Config 2	Units=128, Dense=64, Dropout=0.5	90.47%	90.65%
15	GRU	Config 3	Stacked 64 $\rightarrow$ 32, Dropout=0.4	90.77%	90.92%
16	LSTM	Config 1	Units=64, Dense=32, Dropout=0.3	43.73%	34.66%
17	LSTM	Config 2	Units=128, Dense=64, Dropout=0.5	63.67%	57.15%
18	LSTM	Config 3	Stacked 64 $\rightarrow$ 32, Dropout=0.4	44.40%	35.60%
19	Bidirectional SimpleRNN	Config 1	Units=2 $\times$ 64, Dense=32, Dropout=0.3	92.30%	92.35%
20	Bidirectional SimpleRNN	Config 2	Units=2 $\times$ 128, Dense=64, Dropout=0.5	91.37%	91.43%
21	Bidirectional SimpleRNN	Config 3	Stacked 2 $\times$ 64 $\rightarrow$ 2 $\times$ 32, Dropout=0.4	91.17%	91.19%
22	Bidirectional GRU	Config 1	Units=2 $\times$ 64, Dense=32, Dropout=0.3	92.17%	92.23%
23	Bidirectional GRU	Config 2	Units=2 $\times$ 128, Dense=64, Dropout=0.5	91.03%	91.03%
24	Bidirectional GRU	Config 3	Stacked 2 $\times$ 64 $\rightarrow$ 2 $\times$ 32, Dropout=0.4	91.10%	91.05%
25	Bidirectional LSTM	Config 1	Units=2 $\times$ 64, Dense=32, Dropout=0.3	92.23%	92.29%
26	Bidirectional LSTM	Config 2	Units=2 $\times$ 128, Dense=64, Dropout=0.5	91.87%	91.87%
27	Bidirectional LSTM	Config 3	Stacked 2 $\times$ 64 $\rightarrow$ 2 $\times$ 32, Dropout=0.4	91.47%	91.48%
28	BERT Base	Config 1	LR=2e-5, Epochs=3, Batch=16	95.97%	95.97%
29	BERT Base	Config 2	LR=3e-5, Epochs=3, Batch=16	96.00%	96.00%
30	BERT Base	Config 3	LR=5e-5, Epochs=2, Batch=16	95.87%	95.87%

B Full Confusion Matrices

Figure 7 provides the full normalized confusion matrix heatmaps for all 10 evaluated models on the 3,000-sample test set.



Figure 7: Normalized confusion matrix heatmaps for all 10 models.